

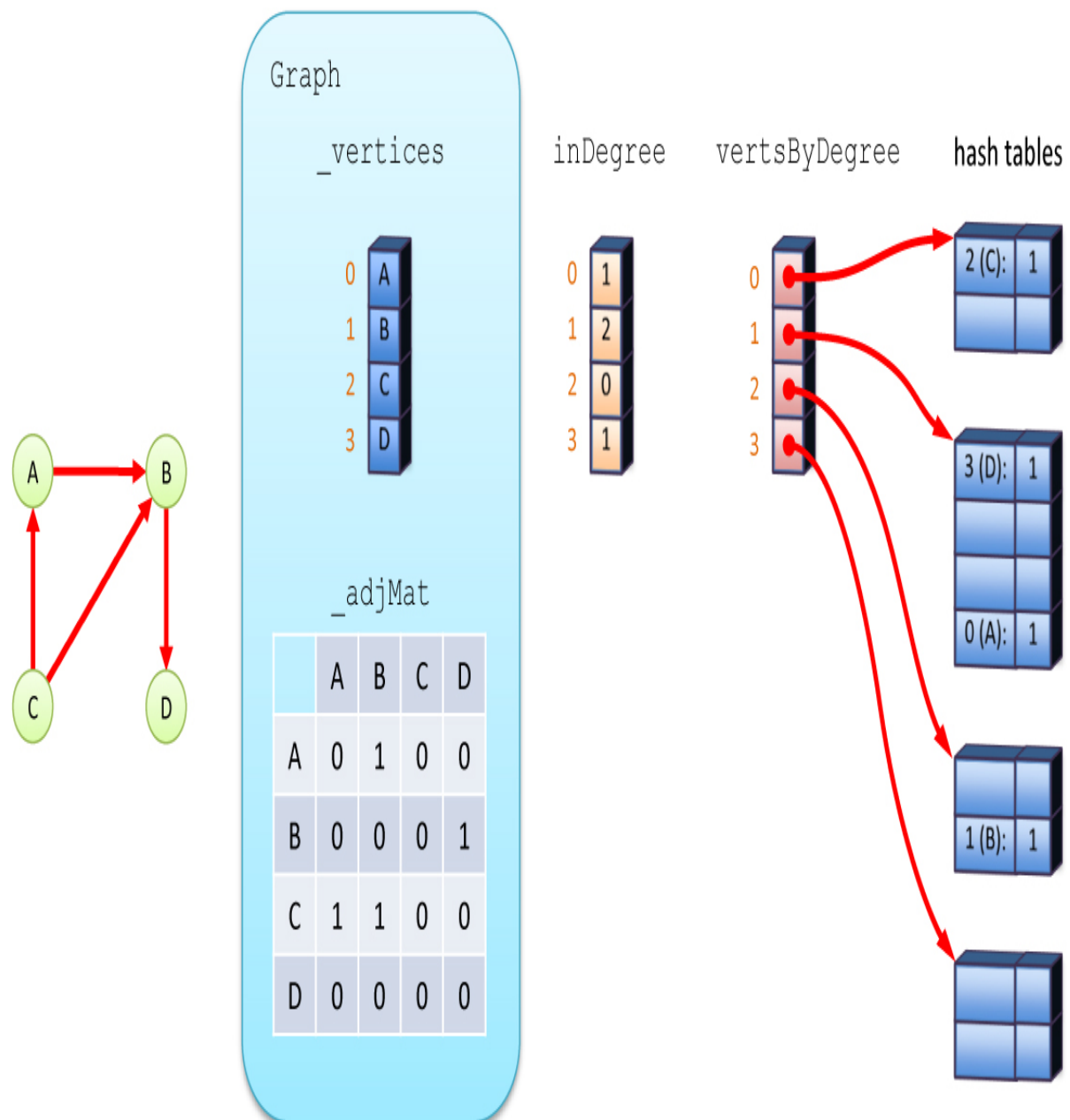


Figure 14-13 Another minimum spanning tree from a fully connected five-node graph

So, what structure should be used to represent the minimum spanning tree? You could try to make a tree structure with an arbitrary number of children for each node, something like the 2-3-4 tree but with any number of children. That could be a good solution, but there's an easier one: use the `Graph` class itself.

A minimum spanning tree is a **subgraph**—a subset of the original graph's vertices and edges. The *spanning* part means that all the vertices in a connected

component are still connected, and the *tree* means there is a single, unique path between every pair of vertices. The trees you've seen so far all share those properties. There was a unique path from the root to every node in the tree, and all of them were reachable from the root.

It's possible for the minimum spanning tree to include every vertex and edge of the input graph. Typically, however, the tree is a proper subset of the vertices and edges forming a connected component.

Python Code for the Minimum Spanning Tree

Our `minimumSpanningTree()` method returns a new graph with a subset of the vertices and edges in the starting graph. We create the MST using a depth-first traversal from a starting vertex. All the vertices in the connected component that includes the starting vertex will go in the MST. Any nonconnected vertices, like A and F in [Figure 14-12](#), are left out. [Listing 14-6](#) shows the code.

Listing 14-6 *The `minimumSpanningTree()` Method of `Graph`*

```
class Graph(object):
...
    def minimumSpanningTree( # Compute a minimum spanning tree
        self, n):           # starting at vertex n
        self.validIndex(n)   # Check that vertex n is valid
        tree = Graph()       # Initial MST is an empty graph
        vMap = [None] * self.nVertices() # Array to map vertex indices
        for vertex, path in self.depthFirst(n):
            vMap[vertex] = tree.nVertices() # DF visited vertex will be
            tree.addVertex(    # last vertex in MST as we add it
                self.getVertex(vertex))
            if len(path) > 1: # If the path has more than one vertex,
                tree.addEdge( # add last edge in path to MST, mapping
                    vMap[path[-2]], vMap[path[-1]]) # vertex indices
        return tree
```

Because the result contains a subset of the original vertices, the vertex indices could be different in the two graphs. For instance, a 10-vertex graph might be composed of two connected components: one with six vertices and the other with four. If you ask for an MST starting from a vertex in the six-vertex component, the MST will have exactly six vertices with indices 0 through 5. Those vertices could have indices up to 9 in the original graph, so you need a

way to track the correspondence of vertices in the original graph to those in the MST.

The `minimumSpanningTree()` method starts with an empty graph—called `tree`—to hold the MST. As we add vertices to it, we note the translation between old and new vertices in an array called `vMap`. The array needs a cell for every possible vertex. Just before we add a vertex to the MST, we know what index it will get because it's placed at the end of the existing vertices. That means the old vertex index maps to the number of the vertices added to the MST so far.

To see how the mapping works, look at the sample `vMap` in [Figure 14-12](#). The `vMap` appears in the upper right of the display as an array aligned with input vertices whose values point to indices in the new tree (vertex indices). As the starting point, vertex C was visited first and becomes the first vertex in the output tree, so the `vMap` shows C mapping to index 0. The next vertex added to the tree is vertex B, so it maps to index 1, and so on. Only the vertices included in the minimum spanning tree will have entries in `vMap` mapping them to their new indices. In [Figure 14-12](#), vertices A and F are not in `vMap` because they are not connected to C.

In the code after setting up the empty `tree` and `vMap`, `minimumSpanningTree()` uses the `depthFirst()` traversal over the graph to visit all the vertices in the connected component. The `depthFirst()` traversal yields both a vertex and the path to the vertex for each visit (in the form of a stack). The first vertex visited will always be the starting vertex, and the path will contain just that one vertex on the visit.

In the depth-first loop body, we first store the mapping from the `vertex` being visited to its index in the MST. The number of vertices already in the `tree` provides the index to store in `vMap`. The next call adds the `vertex` being visited to the MST. Then, if the path to the visited vertex has at least one edge in it, we add the last edge to the MST as well. We get the vertices forming that edge from the last two vertices in the path: `path[-2]` and `path[-1]`. We must translate those vertex indices using the `vMap` array. We're guaranteed that the indices in the path were set in `vMap` because the path contains only vertices that were previously visited by `depthFirst()`.

Does this code handle everything? What about the edges? We add one edge for each vertex added to the MST other than the starting vertex. Is that enough? Yes, adding one edge per vertex means we get exactly the $N-1$ edges from the N vertex connected component. Each added edge is the last edge in the unique

path that leads to that vertex. So, the algorithm covers all the vertices and edges of the connected component that belongs in the MST.

When the depth-first traversal is complete, `tree` contains the full minimum spanning tree, so it is returned to the caller. To help with development and see the detailed contents of the various arrays, you can define some print methods for the `Graph` class, as shown in [Listing 14-7](#).

Listing 14-7 *Methods for Summarizing and Printing Graphs*

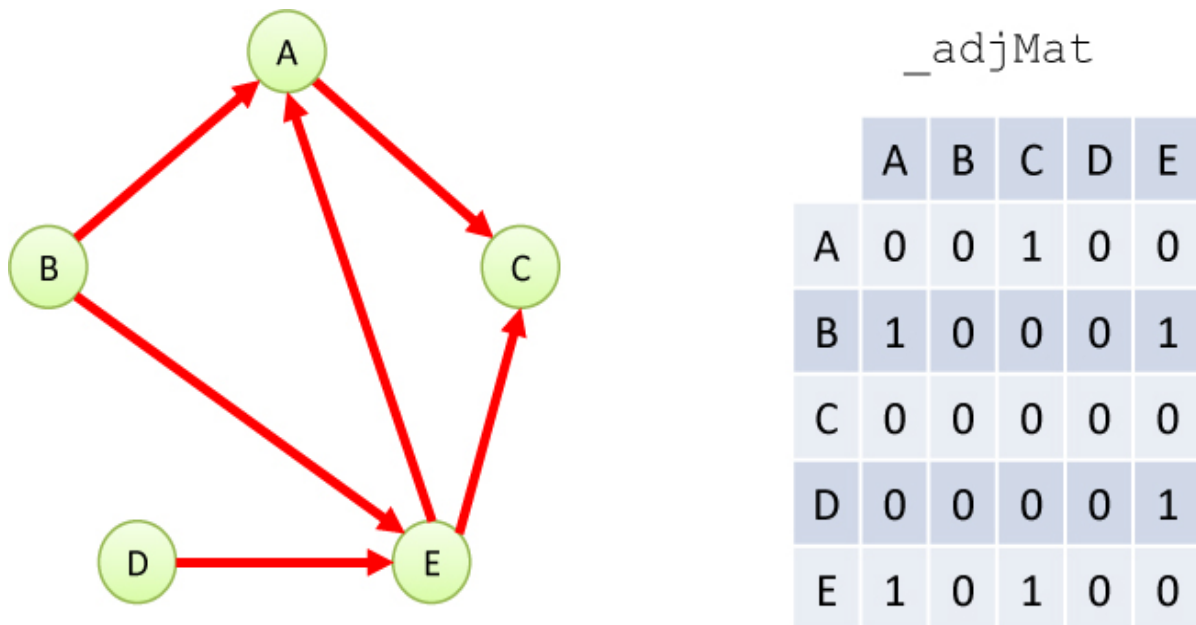
```
class Graph(object):
...
    def __str__(self):          # Summarize the graph in a string
        nVertices = self.nVertices()
        nEdges = self.nEdges()
        return '<Graph of {} vert{} and {} edge{}>'.format(
            nVertices, 'ex' if nVertices == 1 else 'ices',
            nEdges, '' if nEdges == 1 else 's')

    def print(self,            # Print all the graph's vertices and edges
               prefix=''):    # Prefix each line with the given string
        print('{}{}{}'.format(prefix, self)) # Print summary form of graph
        for vertex in self.vertices(): # Loop over all vertex indices
            print('{}{}{}:'.format(prefix, vertex), # Print vertex index
                  self.getVertex(vertex)) # and string form of vertex
            for k in range(vertex + 1, self.nVertices()): # Loop over
                if self.hasEdge(vertex, k): # higher vertex indices, if
                    print(prefix, # there's an edge to it, print edge
                          self._vertices[vertex].name,
                          '<->',
                          self._vertices[k].name)
```

These printing methods use some of Python's string formatting features, which we don't describe here. Instead, we show what they produce on a small example.

For this example, we use a small graph with six vertices, two of which are not connected to the other four. [Figure 14-14](#) shows the original graph on the left along with the depth-first traversal starting at vertex A. The minimum spanning tree is on the right. Vertices B and C do not appear in the minimum spanning tree because they are not connected by an edge to the other vertices. The output of the `print()` method appears below each graph. Note that edges are printed

once next to their “first” (lower index) vertex and not repeated next to their “second” (higher index) vertex.



Starting from a particular vertex,
what other vertices are reachable?

Figure 14-14 *Printed descriptions of a graph and a minimum spanning tree*

The minimum spanning tree algorithm follows the depth-first traversal to get all the vertices in the connected component of the starting vertex. The vertices of other components are ignored. The resulting subgraph will always have $N-1$ edges for the N vertices in the connected component.

How much time does it take to build the minimum spanning tree? Well, it's at least $O(N)$ to go through all the N vertices. The `depthFirst()` traversal method seems as though it should be $O(N)$, as was the case for all the traversal algorithms you've seen for other data structures. A closer inspection, however, shows that for each vertex visited, the `depthFirst()` generator starts up its own internal loop over all vertices to find the first adjacent unvisited one. If there

are few edges, that inner loop could go through most of the vertices before finding the next adjacent one. That means it could take $O(N)$ just to find the next adjacent vertex, and `depthFirst()` traversal could take $O(N^2)$. The breadth-first traversal always completes its inner loop, so it's definitely $O(N^2)$. That's not always going to happen for depth-first, as we discuss in the next chapter, but for now you can assume that worst case and that means the minimum spanning tree takes $O(N^2)$ too.

Topological Sorting

Topological sorting is another operation that can be modeled with graphs. It's useful in situations where items or events must be arranged in a specific order. Let's look at an example.

Dependency Relationships

In school, students find (sometimes to their dismay) that they can't take just any course they want. Some courses have prerequisites—other courses that must be taken first. The course sequence usually models a hierarchy where some concepts cannot be mastered without first understanding another set of concepts. These are **dependency relationships**. They occur everywhere. Algebra depends on understanding arithmetic; linear algebra depends on understanding algebra. In programming, a source file can import or include other source files, which makes one file dependent on another. In cooking, each step depends on completing the previous preparations. For example, baking bread depends on mixing the dough and heating an oven.

Dependency relationships can be described with a graph—specifically a **directed graph**. [Figure 14-15](#) shows a simplified recipe for baking bread.

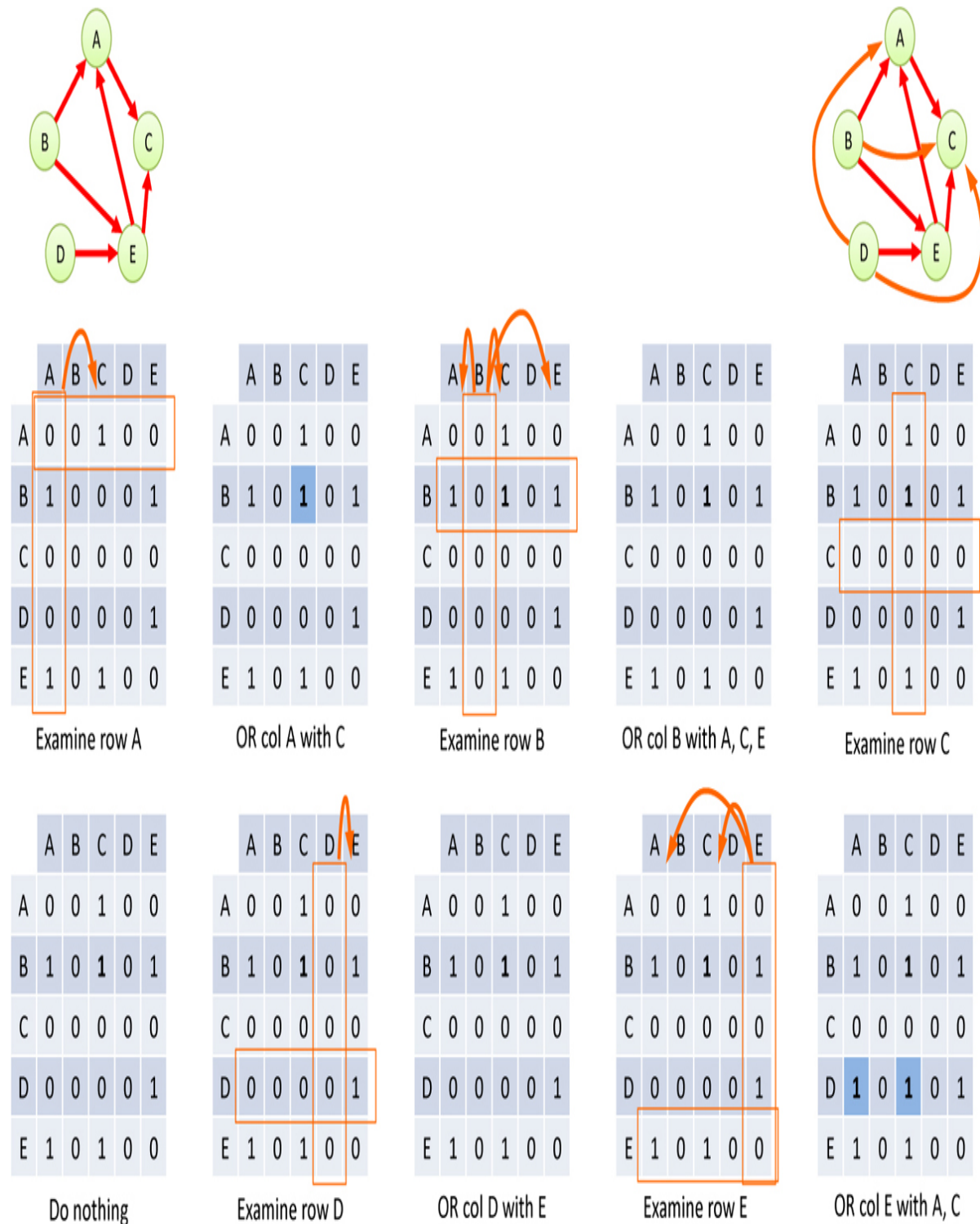


Figure 14-15 A dependency graph for baking bread

To bake bread, you need to have a heated oven and a loaf of dough in a pan. To have a loaf, it should be shaped from dough. That dough should be raised first,

and so on. Most of the steps must be done in a particular order. Removing the eggshells after beating the eggs would be problematic (removing them after raising the dough would be even worse). Some pairs of steps, however, can be done in either order. It doesn't matter which order you measure the flour and the salt; and heating the oven and mixing the dough can be done in either order, even simultaneously. The dependency relations shown by the arrows capture this partial ordering.

Analyzing the different tasks and their relative order may seem trivial to anyone who has baked bread before, but they are anything but trivial to a robot. Imagine a robot that could read a cookbook recipe and figure out all the steps. Assuming it could understand the meaning of the words, it would still need to determine the ordering of tasks. Recipes are usually written in chronological order of the steps, so it could simply follow that sequence. That, however, would probably lengthen the preparation time because steps that could be done at the same time would be done sequentially. Ideally, it would analyze all the possible orderings of the steps and find the most efficient one.

Directed Graphs

Dependency relationships expose a graph feature we haven't discussed yet: the edges need to have a *direction*. When this is the case, the graph is called a *directed graph* or a **digraph**. In a directed graph you can proceed only one way along an edge. The arrows in [Figure 14-16](#) show the direction of the edges.

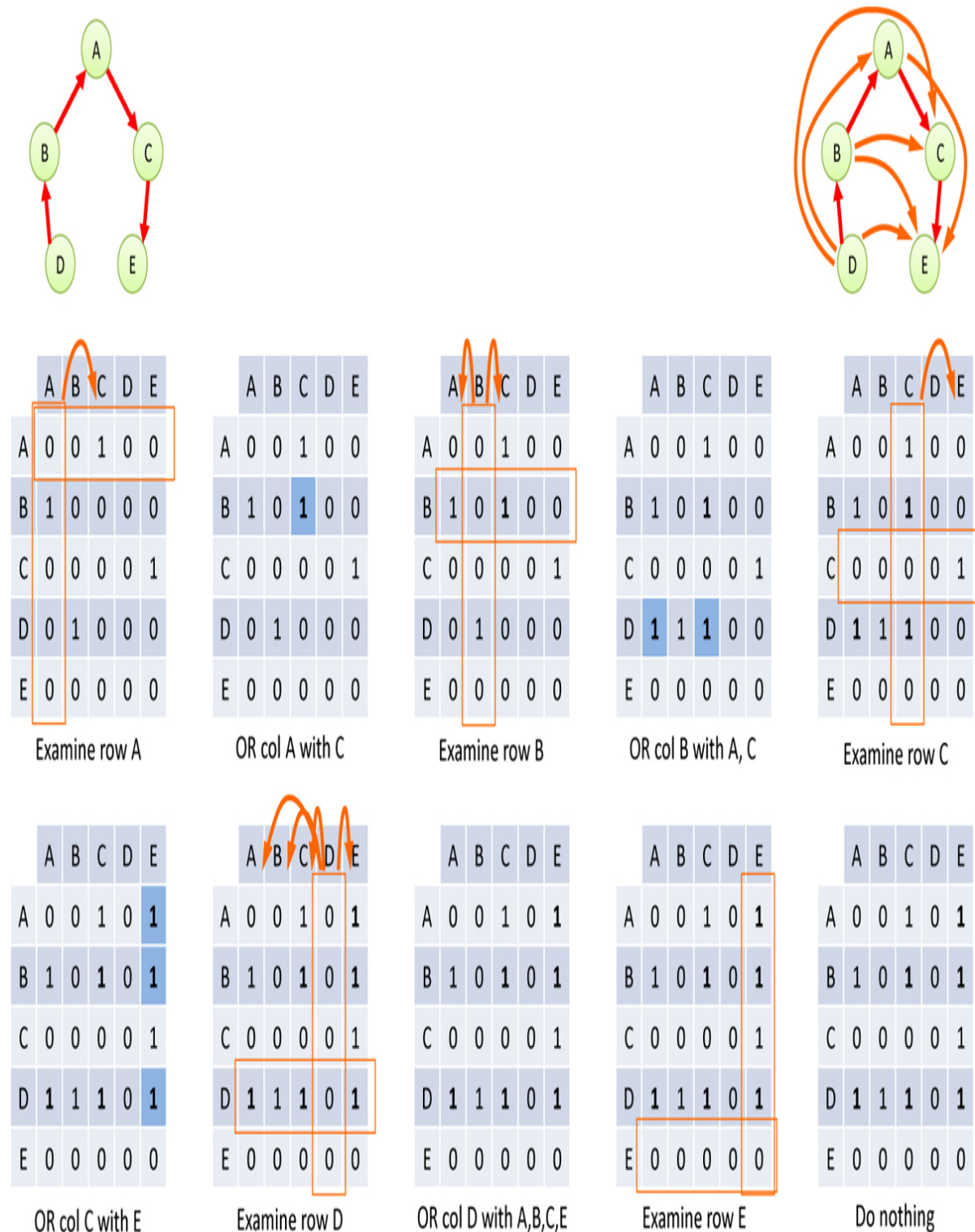


Figure 14-16 *A small directed graph and its adjacency matrix*

In a program, the difference between a undirected graph and a directed graph is that an edge in a directed graph has only one entry in the adjacency matrix.

Figure 14-16 shows a small directed graph and its adjacency matrix.

Each 1 in the matrix represents a single edge. The row labels show where the edge starts, and the column labels show where it ends. Thus, the edge from A to B is represented by a single 1 at row A column B. If the directed edge were reversed so that it went from B to A, there would be a 1 at row B column A instead.

For a undirected graph, as noted earlier, half of the adjacency matrix mirrors the other half, so half the cells are redundant. For directed graphs, however, every cell in the adjacency matrix conveys unique information. The halves are not mirror images.

For a directed graph, the method that adds an edge thus needs only a single statement

```
self._adjMat[A, B] = 1
```

instead of the two statements required in a undirected graph. If you use the adjacency-list approach to represent your graph, then A has B in its list, but—unlike a undirected graph—B does not have A in its list.

Sorting Directed Graphs

Imagine that you make a list of all the actions needed to bake bread, using Figure 14-15 as your input data. You then arrange the actions in the order you need to take them. The final Bake bread step (Bb) is the last item on the list, which might look like this using two initials for each vertex:

Mf,Ms,Ml,Re,Be,Md,Rd,Fl,Pp,Pl,Ho,Bb

Arranged this way, the graph vertices are said to be **topologically sorted**. Any action you must take before some other action occurs before it in the list.

Many possible orderings would satisfy the dependency relationships. You could heat the oven and prepare the pan first as in

Ho,Pp,Mf,Ms,Ml,Re,Be,Md,Rd,Fl,Pl,Bb

This approach also satisfies all the relationships (although the similarity to chemical symbols might be a bit unsettling when thinking about bread). There are many other possible orderings as well. When you use an algorithm to generate a topological sort, the approach you take determines which of various

valid sortings are generated. The specific constraint that must be followed in graph terminology is as follows. For it to be a valid sorting of a directed graph, if there is a path from vertex A to vertex B, vertex A must precede vertex B in the topological sort.

Directed graphs can model other situations besides recipe steps and course prerequisites. Many industrial projects are managed by breaking down the overall project into smaller jobs or tasks. Each task might depend on outputs of other tasks and might require some time to elapse between the end of one task and the start of another. Take building a house as an example. The person who wants the house constructed must identify a site, find a builder, obtain permits, and get financing. The actual construction work depends on having all those tasks done. There could be a delay between completing all the preparation tasks and commencing construction as the builder finds crewmembers available to start the new work. These delays are somewhat predictable, but rarely with any precision.

Modeling job schedules with graphs is called **critical path analysis**. Although we don't show it here, a weighted, directed graph (discussed in the next chapter) can be used, which allows the graph to include the time necessary to complete different tasks in a project. The graph can then tell you such things as the minimum time necessary to complete the entire project and overall costs by looking at different possible topological sorts.

The Graph Visualization Tool

The Graph Visualization tool can model directed graphs too. When the graph has no edges, you can select or clear the Bidirectional checkbox. The checkbox is disabled when any edges exist. You can select New Graph to clear all the vertices and edges and uncheck the Bidirectional box.

The vertices and edges are edited in the same way for directed graphs. The differences are that edges are drawn as a curve with arrow heads, dragging the pointer from one vertex to another only creates the edge from the first one to the second, and clicking a cell in the adjacency matrix only creates the edge from the corresponding row to the corresponding column. [Figure 14-17](#) shows the same directed graph as the bread baking example from [Figure 14-15](#) using the two-letter abbreviations for the operations. The vertices have been rearranged but the edges connecting them remain the same. The adjacency

matrix is large and obscures the table of vertices. Use the collapse and uncollapse button to switch the view of the matrix.

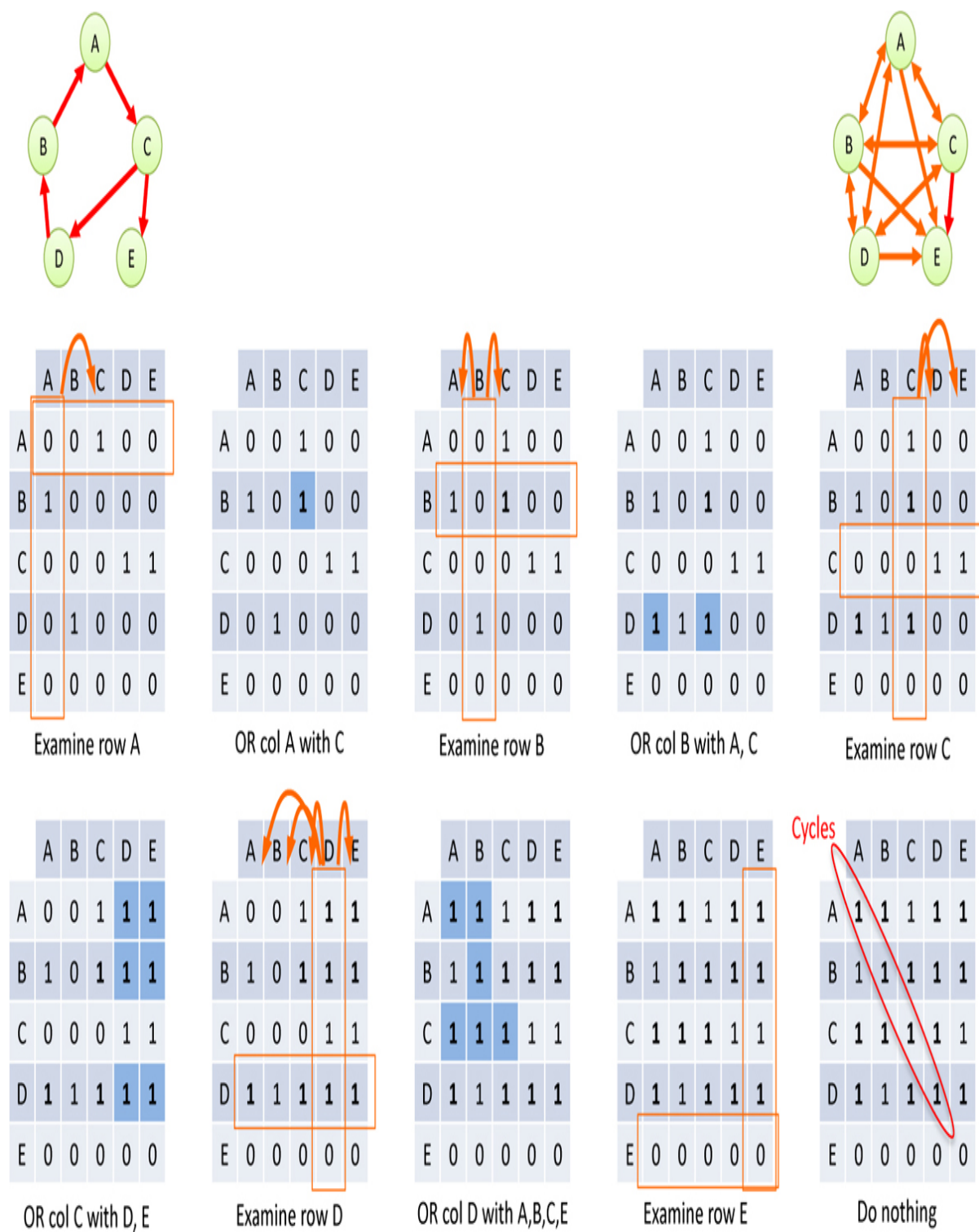


Figure 14-17 A directed graph in the Graph Visualization tool.

You can use the tool to run depth and breadth-first traversals and minimum spanning trees of directed graphs. The results are different than for the undirected graphs because the edges are now directional.

The Topological Sorting Algorithm

Let's look at how to sort the vertices topologically. Here, we start with a basic algorithm and then improve it. For the basic algorithm, we need at least two steps.

Step 1

Find an unvisited vertex that has no (unvisited) predecessors.

The *predecessors* to vertex, V , are those vertices that are directly “upstream” from it—that is, connected to it by an edge that points to V . If an edge points from A to B, then A is a predecessor to B (and B is a *successor* to A). In [Figure 14-16](#), the only vertex with no predecessors is C. In the bread baking example of [Figure 14-17](#), six vertices have no predecessors: Mf, Ms, Ml, Re, Pp, and Ho. These vertices can be found by looking for columns in the adjacency matrix with no edges.

Step 2

Mark the vertex found in Step 1 as visited and add it at the end of the result list.

Steps 1 and 2 are repeated until all the vertices are gone. At this point, the result list holds the vertices arranged in topological order.

You can see the process at work by using the Graph Visualization tool. Construct the graph in [Figure 14-16](#) (or any other graph, if you prefer). Then select the Topological Sort button. There's no need to select a starting vertex because the algorithm doesn't need one.

The animation in the Visualization tool proceeds in two phases. It's a bit fancier than the two-step algorithm shown earlier, but those same steps are used. We look at a couple of different implementations later to see how to sort the vertices efficiently.

Note that this topological sorting algorithm works on unconnected graphs as well as connected graphs. This models the situation in which you have two

unrelated goals, such as getting a degree in mathematics and at the same time taking a hiking trip in the mountains. Some graphs, however, cause problems.

Cycles and Trees

One kind of graph that this basic algorithm cannot handle is a graph with **cycles**. We mentioned how the minimum spanning tree of a graph never creates a loop connecting a vertex to itself by a path of two or more edges. The formal name for such a loop is a *cycle*.

If a path ends up at the vertex where it started by following directed edges, the graph has at least one cycle. In [Figure 14-18](#) the path B-D-C-B forms a cycle. In addition, A-B-D-C-A forms a cycle. Notice that A-B-C-A is not a cycle because you can't go from B to C in this directed graph.



Figure 14-18 *Directed graph with cycles.*

A cycle models the paradox (which some students claim to have encountered at certain institutions), in which course B is a prerequisite for course D, D is a prerequisite for C, and C is a prerequisite for B. Fortunately, such paradoxes are rare.

Cycles can occur in undirected graphs too, but you must be more specific about them. You can typically ignore the cycle formed by following an edge and returning immediately back along the same edge. Cycles must be formed by paths of two or more distinct undirected edges. Multigraphs allow a pair of vertices to be connected by multiple edges, so a cycle could form with as few as two edges. In non-multigraphs, you need at least three undirected edges to form a cycle.

It's easy to figure out whether some kinds of undirected graphs have cycles. If a graph with N nodes has more than $N-1$ edges, it must have cycles. You can make this point clear to yourself by trying to draw a graph with N nodes and N edges that does not have any cycles. On the other hand, detecting cycles in directed graphs and graphs with fewer edges can be hard to do. In [Chapter 15](#), “[Weighted Graphs](#),” the section on efficiency discusses the complexity of finding certain kinds of cycles.

A undirected graph with no cycles is always a tree. It's the presence of cycles that allows two vertices to be connected by distinct paths.

The binary and multiway trees you saw in earlier chapters of this book are types of directed graphs because the edges always link a parent node to a child node. We never allowed those trees to create cycles. Doing so would cause problems like those described for circular lists in [Chapter 5](#).

Topological sorts succeed only on directed graphs with no cycles. Such a graph is called a **directed acyclic graph**, often abbreviated *DAG*. The basic topological sorting algorithm doesn't have a step for what to do if Step 1 fails—when there is no vertex that has no unvisited predecessors. For example, if you ask it to process the graph in [Figure 14-18](#), Step 1 will see that all vertices have at least one predecessor and fail. Thus, we need to add a final step:

Step 3

After Step 1 fails, if the result list has fewer vertices than the graph, the graph must have a cycle.

This three-step algorithm can handle all directed graphs with and without cycles.

Python Code for the Basic Topological Sort

We introduce some new methods in the `Graph` class to work on predecessors and use them to implement the three steps of the basic topological sorting algorithm in [Listing 14-8](#). The first method, `predecessorVertices()`, is a generator like `adjacentVertices()`, which yields all the predecessor vertices of a given vertex, `n`. It validates the vertex index and then loops over all possible indices, yielding only those with an edge pointing to `n`.

Listing 14-8 Code for the Basic Topological Sort of Vertices

```
class Graph(object):
...
    def predecessorVertices( # Generate a sequence of vertex indices
        self, n):          # that are adjacent predecessors to n
        self.validIndex(n)  # Check that vertex index n is valid
        for j in self.vertices(): # Loop over all other vertices
            if j != n and self.hasEdge(j, n): # If other vertex connects
                yield j                     # via edge, yield other vertex index

    def onlyVisitedPredecessors( # Test whether vertex n's predecessors
        self, n, visited):      # have all been visited, if any
        return all(visited[j]   # All predecessors must have been set in
            for j in self.predecessorVertices(n)) # visited array

    def findUnvisitedWithoutPredecessor( # Find a vertex without
        self, visited):               # unvisited predecessor vertices, if any
        for vertex in self.vertices(): # Loop over all vertices
            if (not visited[vertex] and # If vertex is unvisited and has
                self.onlyVisitedPredecessors( # only visited
                    vertex, visited)): # predecessors,
                return vertex              # then return it
        return None                      # Otherwise there's a cycle or no vertices

    def sortVerticesTopologically( # Return a sequence of all vertex
        self):                    # indices sorted topologically
        result = []               # Result list of vertices
        nVertices = self.nVertices() # Number of vertices
        visited = [None] * nVertices # Array to mark visited vertices
        while len(result) < nVertices: # Loop until all vertices handled
            vertex = self.findUnvisitedWithoutPredecessor( # Find an
                visited) # unvisited vertex without predecessors
            if vertex is None: # If no such vertex exists, then raise an
                raise Exception('Cycle in graph, cannot sort') # exception
            result.append(vertex) # Append unvisited vertex and
            visited[vertex] = True # mark it as visited
        return result
```

To implement Step 1, we need a test to find an unvisited vertex with no unvisited predecessors. In other words, the algorithm seeks a vertex where all its predecessors, if any, are among the visited vertices. The `onlyVisitedPredecessors()` method takes a vertex index, `n`, as a parameter and uses Python's `all()` function to test whether all of `n`'s predecessors satisfy

the condition. The arguments to the `all()` function are the values of the `visited` array for each predecessor index. If one or more of those predecessors have a `visited` value of `None` or `False`, then vertex `n` fails the test. Notably, if `n` has no predecessors, the argument list to the `all()` function is empty and it returns `True`.

The `findUnvisitedWithoutPredecessor()` method performs Step 1, looping over all possible vertices, skipping visited vertices, and returning the first unvisited one that satisfies `onlyVisitedPredecessors()`. If it completes the loop without finding an unvisited vertex without predecessors, it returns `None`, signaling that there must be a cycle.

The `sortVerticesTopologically()` method performs Steps 1, 2, and 3. It sets up an empty `result` list to hold the sorted vertices. Next it creates a `visited` array, like the ones used in the traversal methods, marking all the vertices as unvisited. The main work begins with the `while` loop. On each pass of the loop, it tries Step 1 to find a vertex with no predecessors. If that comes back as `None`, it raises an exception to signal the detection of a cycle. Otherwise, it adds the vertex to the end of the `result` and marks it as `visited`, which implements Step 2. If the loop finishes, then there are `nVertices` in the `result` list, and it is returned, handling Step 3.

The basic topological sort algorithm is easy to understand, but let's look at its efficiency. [Figure 14-19](#) shows the different levels of the algorithm. Each level represents one of the methods being called. Let's look at the innermost level first. Every loop through the `vertices()` sequence is, of course, $O(N)$. Thus, one call to the `onlyVisitedPredecessors()` test for a particular vertex looks at all the other vertices, so it takes $O(N)$ time too. At the next level, outward, the `findUnvisitedWithoutPredecessor()` method calls the inner test on every vertex until it finds one that matches. Although that middle level does quit when it finds such a vertex, it still has to scan some fraction of the vertices, so the combined worst-case complexity of the middle level is $O(N^2)$ (to be thorough, we would need to determine the average number of tries needed to find the vertex, but we can assume it's proportional to N without knowing anything about the graph's edges).



Figure 14-19 *Complexity of the basic topological sort algorithm*

The outermost routine, `sortVerticesTopologically()`, calls the middle level until all `nVertices` have been copied. That's another $O(N)$ factor, so each level of the algorithm contributes $O(N)$ as shown. Overall, the algorithm is $O(N^3)$. Can we do better than that?

Improving the Topological Sort

Yes, we can reduce that complexity to at least $O(N^2)$, but it will take a more complicated organization of the data. While this transformation is complicated, learning from examples of how others have improved algorithm performance will help you analyze and improve your own programs. It's another case study in how to select and use the data structures most appropriate for a task.

The key concept that we're interested in is the number of edges for each vertex. If we organize the vertices by the number of inbound edges, we can quickly find the ones with no predecessors. In graphs, the number of inbound and outbound edges are called the **inbound degree** and the **outbound degree**, or **indegree** and **outdegree**, for short.

The first thing we need is a method to get the inbound and outbound degree of a vertex. The `degree()` method in [Listing 14-9](#) takes a vertex as a parameter, counts both kinds of edges at that vertex, and returns the counts as a tuple.

After we know the inbound degree for all the vertices, we can put all the vertices with degree 0 in one place, so they are easy to find. In fact, if we keep an array indexed by degree, we can put each vertex in the cell of the array for its inbound degree. We need some structure in each array cell that can hold a

group of vertices. Should that structure be a stack? A queue? A tree? The answer depends on how we process the vertices, so let's look at the remaining steps.

After we take a vertex out of the cell for degree 0, we can put it in the sorted result list. The successors of that vertex now have one fewer inbound edge, effectively. A successor that had only one inbound edge from the vertex that was put in the result list can now be considered to have degree 0. So instead of tracking which vertices have been visited, we can simply change the degree for each successor vertex, moving it to the proper cell in the array for its new degree. As we process vertices and their successors, every vertex will eventually be moved down to degree 0 (if there are no cycles).

Knowing that we want to move vertices between cells in the array, it becomes clear that we need a structure that enables fast insertion and deletion. Stacks and queues have $O(1)$ insert and delete complexity, but only for specific orders (LIFO and FIFO). For this algorithm we would need to, say, remove vertex 27 from one cell and add it to another.

If we want to insert and delete items by a key, such as their vertex index, the best data structures would be an array or a hash table. Both have $O(1)$ insert and delete times (assuming we don't need to shift items in the array to eliminate holes). The hash table has the added advantage of only needing memory proportional to the items in it; an array would need N cells to hold any of the N vertices.

Figure 14-20 illustrates the relations of structures we need. The left side shows a simple four-vertex graph. Next on its right is the `Graph` object that holds the `_vertices` array and `_adjMat` adjacency matrix to represent it. Next to that is a four-element array that shows the inbound degree for each of the four vertices. For example, vertex A has inbound degree 1 while vertex B has inbound degree 2.



Figure 14-20 *Structures used for topological sort*

The `vertsByDegree` array groups together the vertices by their inbound degree. Each cell in this array references a hash table that holds the keys for the vertices with a particular inbound degree. For example, `vertsByDegree[1]` references a hash table holding two keys. The keys are the vertex indices of vertices of inbound degree one: 0 (A) and 3 (D). The hash table for degree three is empty because none of the vertices have inbound degree three. The hash tables have extra cells to maintain a 50 percent load factor.

The `sortVertsTopologically()` method in [Listing 14-9](#) starts by creating the `vertsByDegree` array. The maximum inbound degree any vertex could ever have is the number of vertices minus one. For a graph with E edges, the highest inbound edge count could be E . To account for vertices with indegree 0, we need to add 1 to E . The size of the array is the minimum of these values: `nVertices` and `nEdges+1`. The `vertsByDegree` array is built using a list comprehension `[{} for j in range(min(self.nVertices(), self.nEdges() + 1))]`. As you saw for building the adjacency matrix, using the list comprehension ensures that N distinct empty hash tables are made to fill the array cells.

Listing 14-9 *The Improved Topological Sort of Vertices*

```
class Graph(object):
...
    def degree(self, n):          # Get degree of vertex as (in, out) pair
        self.validIndex(n)       # Validate vertex index
        inb, outb = 0, 0         # Count inbound and outbound edges
        for j in self.vertices(): # Loop over all vertices
```

```

        if j != n:                # other than target vertex
            if self.hasEdge(j, n): # If other vertex precedes
                inb += 1           # increase inbound degree
            if self.hasEdge(n, j): # If other vertex succeeds n
                outb += 1         # increase outbound degree
        return (inb, outb)        # Return inbound and outbound degree

def sortVertsTopologically( # Return sequence of all vertex indices
    self):                 # sorted topologically more efficiently
    vertsByDegree = [      # Make an empty hash table for every
        {} for j in range( # possible degree, max = nVerts - 1 or
            min(self.nVertices(), self.nEdges() + 1))] # nEdges
    inDegree = [0] * self.nVertices() # Allocate indegree array
    for vertex in self.vertices(): # Loop over all vertices, record
        inDegree[vertex] = self.degree(vertex)[0] # inbound degree
        vertsByDegree[      # In hash table for this inbound degree
            inDegree[vertex]][vertex] = 1 # insert vertex
    result = []             # Result list is initially empty
    while len(              # While there are vertices with inbound
        vertsByDegree[0]) > 0: # degree of 0
        vertex, _ = vertsByDegree[0].popitem() # take vertex out of
        result.append(vertex) # hash table & add it to end of result
        for s in self.adjacentVertices( # Loop over vertex's
            vertex):                   # successors; move them to lower degree
            vertsByDegree[ # In hash table holding successor vertex
                inDegree[s]].pop(s) # delete the successor
            inDegree[s] -= 1 # Decrease inbound degree of successor
            vertsByDegree[ # In hash table for lowered inbound degree
                inDegree[s]][s] = 1 # insert modified successor
    if len(result) == self.nVertices(): # All vertices in result?
        return result                  # Yes, then return it, otherwise cycle
    raise Exception('Cycle in graph, cannot sort')

```

The sorting algorithm creates an `inDegree` array to hold the inbound degree of every vertex. Initially, these are filled with 0s.

Now we can populate the new data structures. The next section is a `for` loop over all the vertices. It calculates the degree of the `vertex` by calling `self.degree()` and extracts the first element of the returned tuple using `[0]` to get the inbound degree.

The sorting algorithm now inserts the `vertex` in the `vertsByDegree` array based on its inbound degree. It finds the appropriate hash table by referencing `vertsByDegree[inDegree[vertex]]`. This kind of nested reference looks a little complicated, but it really is only two array lookups. The `vertex` variable

indexes the `inDegree` array to get a numeric degree for the vertex. That numeric degree indexes the `vertsByDegree` array. The content of that array cell is a hash table. To enter a `key` in a hash table, `ht`, you could write `ht[key] = 1`. That's what's being done by writing `vertsByDegree[inDegree[vertex]][vertex] = 1`; it sets the `vertex` key in the hash table retrieved from `vertsByDegree`. These strings of references can be tricky to understand when you first look; try working from inner to outer references as well as outer to inner.

After the `vertsByDegree` array is populated with all the vertices and an empty `result` list/array created, the algorithm can now process the vertices in a `while` loop. The loop condition tests whether the hash table for inbound degree 0 vertices has any keys in it. If there are no degree 0 vertices, then we're either done, or the graph has at least one cycle in it.

Inside the `while` loop body, we pop one of the degree 0 vertices out of its hash table. Python has several methods for removing items from hash tables. The `popitem()` method removes and returns a key and its value from the hash table. In this case, the sorting algorithm doesn't care which key is removed, nor what value it has, so it puts the value in the underscore (`_`) variable. (Python's `popitem()` always removes the last key inserted, making it behave like a stack.) Another way to remove a key from a hash table is to use the `pop()` method, which requires a specific key.

Next, the algorithm appends the degree 0 vertex to the result list. Because the `vertex` has no predecessors, it can be the next vertex in the output sequence. That allows each of the `vertex`'s successors to reduce their inbound degree by one. The inner `for` loop goes through the adjacent vertices of `vertex`, which are the same as the successors for directed graphs. Each successor vertex, `s`, is removed from the hash table holding `s` based on its current degree. The `vertsByDegree[inDegree[s]]` looks up the hash table and the `.pop(s)` removes the successor's key. Next, the algorithm lowers the degree for `s` by one, and `s` is inserted in the hash table for that lower degree. After all the successors have been lowered by one degree, the algorithm continues the outer `while` loop, popping vertices out of the degree 0 hash table.

When the degree 0 hash table is empty, we check the length of the result list against the number of vertices. If the result contains all the vertices, it can be returned as the sorted vertex list. If not, a cycle prevented finding a vertex without predecessors.

Try looking at the details of a topological sort using the Graph Visualization tool. It implements this improved algorithm and shows how the `inDegree` and `vertsByDegree` arrays are constructed and updated. As the `sortVertsTopologically()` method executes, try to figure out what will happen next at each step. The hash tables are shown as simple lists rather than as arrays with empty cells that would illustrate their load factors, but that is not critical to understanding the overall algorithm.

Efficiency of the Topological Sort

Does this new structure—an array of hash tables—improve the efficiency? Let's review. Building the initial, empty `vertsByDegree` array takes $O(N)$ time. The first `for` loop will also take at least $O(N)$ time because it goes through all the vertices. Now we must look at what happens inside that loop.

The call to `self.degree(vertex)` takes $O(N)$ time because it has to check all N vertices to see whether they have inbound or outbound edges to the given `vertex`. Thus, populating the `inDegree` array makes the overall `for` loop take $O(N^2)$ time because it covers the entire two-dimensional adjacency matrix to compute all of these.

We also must examine the statement that puts the vertices in their appropriate hash table inside the `vertsByDegree` array. It does two array lookups (one inside `inDegree` and the other inside `vertsByDegree`) and a hash table access for every vertex. The lookups and the hash table access are all $O(1)$, so this doesn't add more complexity to the first `for` loop; overall it takes $O(N^2)$ time.

The main `while` loop of the `sortVertsTopologically()` method executes N times (if there are no cycles, and fewer if there are any). Inside the loop, the act of popping an item and inserting it at the end of the result list is $O(1)$. The inner `for` loop must process each of the successors. Finding the (adjacent) successors takes $O(N)$ time, so the inner `for` loop is at least $O(N)$. Some subset of those N vertices will be successors, and each gets popped from a hash table, decremented in degree, and inserted in another hash table, which are all $O(1)$ operations. The inner `for` loop takes $O(N)$ time; thus, the outer, main `while` loop combined with it, takes $O(N^2)$ time.

Both preparing the array of hash tables in the first `for` loop and processing the result in the main `while` loop take $O(N^2)$ time. The extra memory needed for the `vertsByDegree` and `inDegree` arrays has made an improvement over the

$O(N^3)$ time of the basic algorithm. That might seem small, but it can be huge when N grows large.

Connectivity in Directed Graphs

You've seen how, in a undirected graph, you can find all the vertices that are connected by doing a depth-first or breadth-first traversal. When you try to find all the connected vertices in a directed graph, things get more complicated. You can't just start from a randomly selected vertex and expect to reach all the other connected vertices.

Consider the small, directed graph in [Figure 14-16](#) and [Figure 14-20](#). If you start on A, you can get to B and D but not to C. If you start on B, you can get only to D, and if you start on D, you can't get anywhere. The meaningful question about connectivity is: What vertices can you reach if you start on a particular vertex?

The Connectivity Matrix

You can easily use the `depthFirst()` method ([Listing 14-4](#)) to traverse part of the graph starting from each vertex in turn. For the graph of [Figure 14-16](#), the output will look something like this:

```
ABD
BD
CABD
D
```

This is the **connectivity table** for the directed graph. The first letter is the starting vertex, and subsequent letters show the vertices that can be reached (either directly or via other vertices) from the starting vertex.

Transitive Closure and Warshall's Algorithm

In some applications it's important to find out quickly whether one vertex is reachable from another vertex. This is the essential question in genealogy: Who are my ancestors? Another example is a celebrity connection game such as the "six degrees of Kevin Bacon," where people try to find a path through acquaintances to reach a particular person. Some people hypothesize that you need at most a path length of six acquaintances to reach a celebrity. In many

applications the path length doesn't matter. Perhaps you want to take a trip by train from Athens to Kamchatka, and you don't care how many intermediate stops you need to make. Is this trip possible? Graphs are ideal for answering these questions. The trip won't be possible unless the vertices are part of the same connected component.

You could examine the connectivity table, but then you would need to look through all the entries on a given row, which would take $O(N)$ time (where N is the average number of vertices reachable from a given vertex). But you're in a hurry. Is there a faster way?

It's possible to construct a table that will tell you quickly (that is, in $O(1)$ time) whether one vertex connects to another. Such a table can be obtained by systematically modifying a graph's adjacency matrix. The graph represented by this revised adjacency matrix is called the **transitive closure** of the original graph. Such a revised matrix can be called the **connectivity matrix**.

Remember that in an ordinary adjacency matrix the row number indicates where an edge starts, and the column number indicates where it ends. The connectivity matrix has a similar arrangement, except the path length between the two vertices may be more than one. In the adjacency matrix, a 1 or True at the intersection of row C and column D means there's an edge from vertex C to vertex D , and you can get from one vertex to the other in one step. Of course, in a directed graph it does not follow that you can go the other way, from D to C .

You can use **Warshall's algorithm** named for Stephen Warshall to find the transitive closure of the graph. It changes the adjacency matrix into a connectivity matrix. This algorithm does a lot in a few lines of code. It's based on a simple idea:

If you can get from vertex L to vertex M , and you can get from M to N , then you can get from L to N .

A two-step path is thus derived from two one-step paths. The adjacency matrix shows all possible one-step paths, so it's a good starting place to apply this rule.

You might wonder whether this algorithm can find paths of more than two edges. After all, the rule only talks about combining two one-edge paths into one two-edge path. As it turns out, the algorithm can build on previously discovered multiedge paths to create paths of arbitrary length. The basic idea is: if you can build the connectivity matrix for one and two edge paths, then you

could apply the same algorithm to that table to build all the three-edge and fewer paths. If you keep reapplying the algorithm, you will eventually discover all the possible paths.

Here's how it works. Let's use the adjacency matrix of [Figure 14-21](#) as an example. For this example, you examine every cell in the adjacency matrix, one row at a time.



Figure 14-21 *A five-vertex directed graph and its adjacency matrix*

Row A

Let's start with row A. There's nothing (0) in columns A and B of that row, but there's a 1 at column C, so you can stop there.

Now the 1 at this location says there is a path from A to C. If you knew there was a path from some other vertex X to A, then you would know there was a path from X to C. Where are the edges (if any) that end at A? They're in column A. So, you examine all the cells in column A. In the `_adjMat` of [Figure 14-21](#), there's only one 1 in column A: at row B. It says there's an edge from B to A. So, you know there's an edge from B to A, and another (the one you started with when examining row A) from A to C. From this, you infer that you can get from B to C in two steps. You can verify this is true by looking at the graph.

To record this result, you put a 1 at the intersection of row B and column C. The result is shown in the second matrix in [Figure 14-22](#). The highlighted cell shows where the value changed to 1.



Figure 14-22 *Steps in Warshall's algorithm*

The remaining cells of row A are blank. You only need to copy the contents of column A to columns that have a 1 in row A. In other words, you perform an operation akin to a bitwise OR of column A and column C for this first “bit” that you find turned “on”. Note that column A also has a 1 for vertex E. Column C already had a 1 for vertex E, so the bitwise OR operation doesn’t change it.

Rows B, C, and D

Next, you go to row B. The first cell, at column A, has a 1, indicating an edge from B to A. Do any edges end at B? You look in column B, but it’s empty, so you know that none of the 1s you find in row B will result in finding longer paths because no edges end at B. You could perform the bitwise OR of column B with the three columns that have a 1 in row B, but a bitwise OR with 0s doesn’t change anything, as shown in the fourth panel of [Figure 14-22](#).

Row C has no 1s at all, so you go to row D. Here, you find an edge from D to E. Column D is empty, however, so no edges end on D and the OR changes nothing.

Row E

In row E you see there are edges from E to A and from E to C. Looking in column E, you see the first entry is for the edge B to E, so with B to E and E to A, you infer there’s a path from B to A. That path, however, has already been discovered, as indicated by the 1 at that location.

There's another 1 in column E, at row D. This edge from D to E plus the ones from E to A and E to C imply paths from D to A and C, so you insert a 1 in both of those cells. The result is shown in last matrix of [Figure 14-22](#).

Warshall's algorithm is now complete. You've added three 1s to the adjacency matrix, which now shows which nodes are reachable from another node in any number of steps. The graph at the top right of [Figure 14-22](#) shows the edges added to the graph by the transitive closure as curved arrows in a different color.

Long Paths

Can Warshall's algorithm find long paths in the graph and build the full closure? It seems as though going row by row once through the matrix might not find long, complicated chains of edges. [Figure 14-23](#) shows a longer example.



Figure 14-23 *Warshall's algorithm on a long path*

In this graph, the five vertices are connected by four edges to form a long chain. Each step of Warshall's algorithm adds new edges to the matrix. First, one edge is added for row A, then two edges when it examines row B, and then three edges for row C. The new entries sometimes line up in a column and sometimes in a row. The six edges are all that are needed to build the final connectivity matrix; rows D and E contribute nothing because either the row or its corresponding column contains all 0s.

Cycles

If D were connected to E in the graph of [Figure 14-23](#), it could form a cycle, assuming the direction of the edge went from E to D. What happens in the connectivity matrix if there's a cycle? Does Warshall's algorithm still work?

In fact, Warshall's algorithm can be used to *detect cycles*. Consider the example shown in [Figure 14-24](#). Vertices A, B, C, and D form a cycle with vertex E dangling from vertex C. In the initial adjacency matrix, which is the upper-left matrix in the figure, the identity diagonal where the row index is the same as the column index has all 0s. That's what you expect in adjacency matrices because only pseudographs to allow vertices to have edges to themselves.



Figure 14-24 *Warshall's algorithm applied to a graph with a cycle*

If you go through the steps of Warshall's algorithm, the final connectivity matrix shows all 1s along the diagonal except for vertex E (as circled in the matrix at the bottom right of [Figure 14-24](#)). That means vertices A, B, C, and D must be part of at least one cycle. There is some path starting from each vertex that leads back to itself.

The presence of 1s along the diagonal tells you that cycles must exist, but not how many there are nor which vertices are in which cycles. Those are harder problems to solve. Nevertheless, it's useful to have methods that identify the presence (and absence) of cycles. The 0 in the diagonal entry for vertex E in [Figure 14-24](#) tells you that it is not part of a cycle. (Perhaps schools should be required to check for 0s by applying this to the dependency relationship graph of all their course prerequisites.)

Implementation of Warshall's Algorithm

One way to implement Warshall's algorithm is with three nested loops (as suggested by Sedgewick; see [Appendix B](#), "Further Reading"). The outer loop looks at each row; let's call its variable R . The loop inside that looks at each cell (column) in the row; it uses variable C . If a 1 is found in matrix cell (R, C) , there's an edge from R to C , and then a third (innermost) loop is activated.

The third loop performs the OR operation between column R and column C . It has to loop over each of the cells (vertices) in those columns using its own variable and perform the OR between their values. We leave the details as an exercise.

Because there are three loops over all N vertices, the overall complexity is $O(N^3)$. That's a lot of computation to build the connectivity matrix. If you only want to find the answer to the problem "Is there a sequence of train trips that go from Athens to Kamchatka?" you could find the answer in $O(N^2)$ time with a depth-first or breadth-first search. Building the full connectivity matrix first, however, can make a huge difference in other, advanced graph algorithms. It could also be used to test for the presence of cycles, possibly returning as soon as any diagonal is set to one. That would still take $O(N^3)$ time (in the worst case and average case), so it's not very fast. We discuss the complexity of this and other graph algorithms in the next chapter.

Summary

- Graphs consist of vertices connected by edges.
- Graphs can represent many real-world entities such as transportation routes, electrical circuits, and job scheduling.
- Vertices are adjacent if a single edge connects them.
- The adjacency of vertices is usually represented by either an adjacency matrix or adjacency lists.
- Adjacency matrices can be represented using two-dimensional arrays or hash tables.
- Traversal algorithms allow you to visit each vertex in a graph in a systematic way and are the basis of several other activities such as searches.

- The two main traversal algorithms are depth-first (DF) and breadth-first (BF).
- The depth-first traversal can be based on a stack; the breadth-first traversal can be based on a queue.
- A breadth-first search finds the shortest path between two vertices (in terms of number of edges), if one exists.
- Depth-first search explores parts of the graph furthest away from the starting vertex early in the traversal, which can be useful in move analysis of games.
- A minimum spanning tree (MST) is a subgraph with the minimum number of edges necessary to connect all a undirected graph's vertices.
- Minimum spanning trees are useful in finding layouts of networks with the fewest number of interconnections.
- A minimum spanning tree can be determined using depth-first traversal on an unweighted, undirected graph.
- Trees are a type of undirected graph where a unique path connects any two vertices.
- In a directed graph, edges have a direction (often indicated by an arrow).
- Directed graphs can represent situations such as dependency relationships, river flows, and one-way road networks.
- The adjacency matrices of undirected graphs always have mirror image symmetry, but those of directed graphs do not.
- In a topological sort of directed graph vertices, if there is a path from vertex A to vertex B, vertex A precedes B in the result list. Vertex pairs not connected by a path can appear in either order.
- Topological sorting can be done only on directed acyclic graphs (DAG) —graphs without cycles.
- Topological sorting is typically used for scheduling complex projects that consist of tasks contingent on other tasks.

- Topological sorting can be done in $O(N^2)$ time, where N is the number of vertices, by computing the inbound degree of each vertex and holding the vertices in hash tables for each degree.
- Warshall's transitive closure algorithm finds whether there is a connection, of either one or multiple edges, from any vertex to any other vertex.
- Warshall's algorithm transforms an adjacency matrix into a connectivity matrix, and that matrix can be used to detect the existence of cycles.
- The basic implementation of Warshall's algorithm could take $O(N^3)$ time.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. In a graph, a(n) _____ connects two _____.
2. How do you tell, by looking at its adjacency matrix, how many edges there are in an undirected graph?
3. In a game simulation using a graph, a(n) _____ corresponds to a game board state and a(n) _____ corresponds to a player's move.
4. A directed graph is one in which
 - a. you must follow the minimum spanning tree.
 - b. you must go between vertices in topologically sorted order.
 - c. you can go in only one direction from one given vertex to another.
 - d. you can go in only one direction on any valid path.
5. If a graph's adjacency matrix has rows $[0,1,0,0]$, $[1,0,1,1]$, $[0,1,0,0]$, and $[0,1,1,0]$, what is the corresponding adjacency list for vertices A, B, C, and D?
6. A minimum spanning tree is a graph in which
 - a. the number of edges connecting all the vertices is as small as possible.
 - b. the number of edges is equal to the number of vertices.

- c. all unnecessary vertices have been removed.
 - d. every combination of two vertices is connected by the minimum number of edges.
7. How many different minimum spanning trees are there in a undirected graph of three vertices and three edges?
8. Choose the fastest way to check whether a path exists from vertex A to vertex Z in a directed graph among these options.
- a. Get the `minimumSpanningTree(A)`, and then find the path from the root of that tree to Z.
 - b. Loop over the vertices returned by calling `depthFirst(A)` until Z shows up.
 - c. Loop over the vertices returned by calling `breadthFirst(Z)` until A shows up.
 - d. Apply Warshall's algorithm to the graph and then check the connectivity matrix to see if A can reach Z.
9. A undirected graph must have a cycle if
- a. any vertex can be reached from some other vertex.
 - b. the number of connected components is more than one.
 - c. the number of edges is equal to the number of vertices.
 - d. the number of paths is fewer than the number of edges.
10. A(n) _____ is a graph with no cycles.
11. The degree of a vertex
- a. is the number of edges in the path linking it to a starting vertex.
 - b. is the number of edges that connect it to other vertices.
 - c. is the number of vertices in its connected component of the graph.
 - d. is half the number of edges in its row of the adjacency matrix.
12. Can a minimum spanning tree for a undirected graph have cycles?
13. True or False: There may be many correct topological sorts for a given directed graph.
14. Topological sorting results in

- a. edges being directed so vertices are in ascending order.
 - b. vertices listed in order of increasing number of edges from the beginning vertex.
 - c. vertices arranged in ascending order, so F precedes G, which precedes H, and so on.
 - d. vertices listed so the ones later in the list are downstream from the ones earlier.
15. If a graph's adjacency matrix has rows $[0,1,0,0]$, $[0,0,0,1]$, $[1,0,0,0]$, and $[1,0,0,0]$ with vertices A, B, C, and D, could it be passed as an argument to `sortVertsTopologically()`? If so, what would the result be?
16. What's a DAG?
17. Warshall's algorithm
- a. finds the largest cycle in a graph, if there is one.
 - b. changes the adjacency matrix into a connectivity matrix.
 - c. sorts the vertices in ascending order but not topologically.
 - d. finds the fewest number of edges needed to perform closure.
18. Under what conditions does it make sense to perform a topological sort on a undirected graph?
19. If graph G1 has 100 vertices and G2 has 10 vertices, what's the computing time ratio between calling `sortVertsTopologically()` on G1 and G2?
20. Which algorithm solves the bridges of Königsberg problem?
- a. Warshall's algorithm
 - b. the minimum spanning tree algorithm
 - c. the topological sort algorithm
 - d. the breadth-first traversal algorithm

Experiments

Carrying out these experiments will help to provide insights into the topics covered in the chapter. No programming is involved.

14-A You saw how depth-first traversal is used to determine the minimum spanning tree of a particular graph. Could you use breadth-first traversal instead? If not, why not? If so, what would be different between the two methods? Experiment with the sample graphs shown in this chapter.

14-B Think about representing the bridges of Königsberg network (see [Figure 14-3](#)) in the computer. Would using an adjacency matrix work? How about an adjacency list? Does it matter if the bridges need distinct labels (like those in the figure) or if all that needs to be stored is the number of edges between two vertices? If either representation won't work, explain why not, and propose a way to make it work.

14-C Using the Graph Visualization tool, start with a new (empty graph) and collapse the Adjacency Matrix window. Then randomly fill in 5 vertices and add 7 edges. Without exposing the Adjacency Matrix, write down the adjacency matrix for the graph. When you're done, expand the matrix view to see if you got it right. Repeat this exercise with a directed graph of 5 vertices and 10 edges.

14-D On paper, create a five-by-five matrix. Put Xs along the diagonal. Then randomly fill some of the cells with 1s, leaving the rest blank (or filled with 0s) to make a five-vertex adjacency matrix. Don't worry about symmetry around the diagonal. Now, with the Adjacency Matrix of the Graph Visualization tool hidden, create the corresponding directed graph from your paper matrix. When you're done, show the matrix in the tool to see if the graph corresponds to your adjacency matrix.

14-E In the Graph Visualization tool, see whether you can create a directed graph with a cycle that the Topological Sort operation cannot identify.

Programming Projects

Writing programs to solve the Programming Projects helps to solidify your understanding of the material and demonstrates how the chapter's concepts are applied. (As noted in the Introduction, qualified instructors may obtain completed solutions to the Programming Projects on the publisher's website.)

- 14.1 Change the `Graph` class to be able to create both kinds of graphs: undirected or directed. Add an optional `directed` parameter to the constructor that is `False` by default. Change all the methods that have different behavior for directed graphs than undirected graphs based on whether the `directed` flag is set, including the `print()` method to show `->` (instead of `<->`) for directed edges. Construct both kinds of graphs using 10 vertices (A, B, C, D, E, F, G, H, I, and J) and the 12 edges (AG, AI, CF, DA, DI, HD, HE, HF, HG, IH, JC, and JH). Show the output of the `print()` method on both graphs and the `depthFirst()` traversal vertices and paths starting from vertex J.
- 14.2 Make a recursive traversal generator, `depthFirstR()`. The method shown in [Listing 14-4](#) uses a stack to track the path to the current vertex. Your method should make use of the call stack and values passed in the recursive calls to determine the order of visiting the vertices and the paths to yield. Demonstrate the output of your generator on the graph in Project 14.1 and on the graph used in the second breadth-first example in [Figure 14-10](#). Show the order of the vertices and the paths to each one.
- 14.3 Modify the `breadthFirst()` generator to return the path to each vertex as well as the vertex being visited. Use it to write a `shortestPath()` method that finds the shortest path between two vertices, if such a path exists. Demonstrate its output searching for the shortest path from vertex A to H in the second breadth-first example of [Figure 14-10](#), and from vertex F to A in the initial graph of [Figure 14-14](#). Include a full breadth-first traversal starting from vertices A and F, respectively, in those graphs.
- 14.4 Implement Warshall's algorithm to compute the connectivity matrix from the adjacency matrix of a graph. Instead of updating the adjacency matrix, write a new method, `connectivityMatrix()`, that returns a new matrix. The result can start out as a copy of the internal `_adjMat` (using the Python `dict`'s `copy()` method). Use your directed graph implementation from Project 14.1. Write a second method, `hasCycles()`, that tests for the presence of cycles in the graph. Demonstrate your methods on the graphs of [Figure 14-22](#), [Figure 14-23](#), and [Figure 14-24](#).
- 14.5 A *clique* is a graph or subgraph of N vertices where every vertex is adjacent to all the $N-1$ other vertices. In graphs of communication

patterns, cliques can indicate interconnected and influential groups. They represent groups of people or organizations that communicate with one another—more so than with others in the graph. That tendency to interact only with members of the group leads to the feeling of exclusion felt by people outside a social clique.

Add a method to the `Graph` class that returns all the subgraph cliques with `N` vertices. The parameter, `N`, should be between 2 and the total number of vertices in the graph to be meaningful. Every pair of adjacent vertices forms a clique of size 2 (in a undirected graph), so there is exactly one subgraph clique of size 2 per edge. Larger cliques can be formed by adding a single vertex to a smaller one, if that vertex has edges to all the vertices in the smaller clique.

Your method should return the cliques as subgraphs with `N` vertices. Demonstrate your method's output when seeking cliques of size 3, 4, and 5 in at least the following two graphs:

- A 10-vertex graph where five of the vertices are fully interconnected and the other five are not connected to more than one other vertex.
- A 10-vertex graph with three overlapping cliques of size 4. The cliques overlap by sharing one vertex. This pattern can be generated using the following expression for vertex index pairs:

```
[(a, b) for c in range(0, 9, 3)
 for a in range(c, c + 4) for b in range(a + 1, c + 4)]
```

Note: Finding cliques in graphs can be quite computationally complex, especially when searching for all of them. Running your method on large graphs could take a long time to process, as we discuss in the next chapter.

14.6 The Knight's Tour is an ancient and famous chess puzzle. The object is to move a knight from one square to another on an otherwise empty chess board until it has visited every square exactly once. Write a program that solves this puzzle using a depth-first search. It's best to make the board size variable so that you can attempt solutions for smaller, square boards ($K \times K$). The regular 8×8 board could take years to solve on small computers, but a 5×5 board should take less than a minute. We have more to say about the complexity of this problem in the next chapter.

Refer to the “[Depth-First Traversal and Game Simulations](#)” section in this chapter, keeping in mind that a puzzle is like a one-player game. It may be easier to think of a new knight being created and remaining on the new square when a move is made (rather than moving a single knight around). This way, a sequence of added knights represents the game board state, and the occupied squares can be deduced from the knights’ positions. When the board is completely filled with knights (the sequence of knights equals the size of the board), you win.

When looking for its next move, a knight must not only make a legal knight’s move (two spaces in one direction and one space in the other), but it must also not move off the board or onto an already-occupied (visited) square. If you make the program display the board and wait for a keypress after every move, you can watch the progress of the algorithm as it places more and more knights on the board. When it gets boxed in, you can see it backtrack by removing some knights and trying a different series of moves.

This problem has some complexities that might not seem obvious. One of the most important is what the vertices in the graph represent. Looking at the tic-tac-toe boards in [Figure 14-8](#) would suggest using vertices to represent the board states: where each of the nine squares is either blank, an X, or an O. To use the `depthFirst()` traversal method shown in [Listing 14-4](#), you would need to first create a `Graph` with a vertex for every possible board state. In tic-tac-toe there are $3^9 = 19,683$ possible ways of placing blank, X, or O in the nine squares. Even though many of those would be impossible in a real game (for example, where the number of Xs and Os differ by more than 1), creating all those vertices and then adding edges between them would be time-consuming. For an 8×8 chessboard where every square is either empty or occupied by a knight, there are 2^{64} , or over 4 billion, board states. That is not likely to be an efficient way to solve this problem.

For many game simulations, the graph is not fully created at the beginning. As moves are made and potential counter moves are explored, new vertices are added to the graph based on the legal moves from the last board state. Thus, the graph is only partially represented throughout the game. That means that the depth-first traversal methods we implemented won’t work to solve the Knight’s tour because neither the adjacency matrix nor the list of vertices is complete.

Another approach is to create *one vertex per square on the board*. Edges between vertices could then represent legal knight moves in chess. In this way the graph represents *legal board moves*, not board states. The state of the board

is implicit in the path taken by the search, which is a sequence of squares where the knights are placed. The number of vertices for the graph would be $K \times K$, with approximately $4 \times K \times K$ edges. If you use the `depthFirst()` traversal method on this legal board move graph, would you solve the puzzle? This method would certainly provide a path of all legal moves and avoid revisiting vertices (squares) previously visited on the path. The problem, however, is that it is designed to visit every vertex exactly once.

In the case of the Knight's Tour, you need to explore every possible *path* to a vertex. To see why, imagine that after visiting 24 of the 25 squares in a 5×5 board, you find that the last empty square cannot be reached from the last square visited. So, you must backtrack in the depth-first search. Let's say you return to the 20th knight and try a new path through the remaining 5 squares. If you had marked 4 of those last 5 squares in the `visited` array used by the `depthFirst()` traversal method, they wouldn't be searched again. You need a different way of marking what has already been searched.

If you use a legal board move graph, you will need to write a depth-first traversal that explores all potential paths in the graph, not just all vertices. If you create game board state vertices as you search, then you must ensure that you visit those vertices in depth-first order.

15. Weighted Graphs

In This Chapter

- [Minimum Spanning Tree with Weighted Graphs](#)
- [The Shortest-Path Problem](#)
- [The All-Pairs Shortest-Path Problem](#)
- [Efficiency](#)
- [Intractable Problems](#)

In the preceding chapter you saw that a graph's edges can have direction. This chapter explores another edge feature: weight. For example, if vertices in a weighted graph represent cities, the weight of the edges might represent distances between the cities, or costs to fly between them, or the number of automobile trips made annually between them (a figure of interest to highway engineers).

When you include weight as a feature of a graph's edges, some interesting and complex questions arise. What is the minimum spanning tree for a weighted graph? What is the shortest (or cheapest) distance from one vertex to another? Such questions have important applications in the real world.

We first examine a weighted but undirected graph and its minimum spanning tree. In the second half of this chapter, we examine graphs that are both directed and weighted, in connection with the famous Dijkstra's algorithm, used to find the shortest path from one vertex to another.

Minimum Spanning Tree with Weighted Graphs

To introduce weighted graphs, we return to the question of the minimum spanning tree. Creating such a tree is a bit more complicated with a weighted graph than with an unweighted one. When all edges are the same weight, it's